



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7

по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Школьный И.В.*

(подпись)

Преподаватель *Трушин С.М.*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Метод решения	4
1.1 Задание 1. Отбор признаков по корреляции и простым критериям	6
1.2 Задание 2. Сравнение методов отбора на ансамблевых моделях	8
1.3 Интерпретируемый отбор признаков.....	12
Вывод.....	16

ВВЕДЕНИЕ

Цель практической работы №7 — освоение методов отбора признаков в задачах машинного обучения, сравнение их эффективности и интерпретируемости, а также анализ их влияния на качество моделей.

1 МЕТОД РЕШЕНИЯ

Необходимо подобрать датасет, в котором не менее 1000 строк, хотя бы 4 числовых признака и 2 категориальных. Импорт представлен в Листинге 1.

Листинг 1 — Импорт библиотек и датасета

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

df = pd.read_csv('ship.csv')
df.head()
```

Так выглядят первые 5 строк датасета (Рисунок 1).

	Date	Ship_Type	Route_Type	Engine_Type	Maintenance_Status	Speed_Over_Ground_knots	Engine_Power_kW	Distance_Traveled_nm	Draft_meters	Weather_Condition
0	2023-06-04	Container Ship	NaN	Heavy Fuel Oil (HFO)	Critical	12.597558	2062.983982	1030.943616	14.132284	Moderate
1	2023-06-11	Fish Carrier	Short-haul	Steam Turbine	Good	10.387580	1796.057415	1060.486382	14.653083	Rough
2	2023-06-18	Container Ship	Long-haul	Diesel	Fair	20.749747	1648.556685	658.874144	7.199261	Moderate
3	2023-06-25	Bulk Carrier	Transoceanic	Steam Turbine	Fair	21.055102	915.261795	1126.822519	11.789063	Moderate
4	2023-07-02	Fish Carrier	Transoceanic	Diesel	Fair	13.742777	1089.721803	1445.281159	9.727833	Moderate

Рисунок 1 — Первые 5 строк датасета

Сам датасет имеет 18 признаков и 2736 строк.

Посмотрим на наличие пропусков в датасете (Рисунок 2).

```
Date          0
Ship_Type      136
Route_Type     136
Engine_Type    136
Maintenance_Status 136
Speed_Over_Ground_knots 0
Engine_Power_kW    0
Distance_Traveled_nm 0
Draft_meters      0
Weather_Condition 136
Cargo_Weight_tons 0
Operational_Cost_USD 0
Revenue_per_Voyage_USD 0
Turnaround_Time_hours 0
Efficiency_nm_per_kWh 0
Seasonal_Impact_Score 0
Weekly_Voyage_Count 0
Average_Load_Percentage 0
dtype: int64
```

Рисунок 2 — Пропуски в датасете

Замечаем, что сразу в пяти признаках обнаружено по 136 пропусков. Заполним их самым часто встречающимся значением, то есть модой (Листинг 2).

Листинг 2 — Заполнение пропусков

```
df['Route_Type'].fillna(df['Route_Type'].mode()[0], inplace=True)
df['Ship_Type'].fillna(df['Ship_Type'].mode()[0], inplace=True)
df['Engine_Type'].fillna(df['Engine_Type'].mode()[0], inplace=True)
df['Maintenance_Status'].fillna(df['Maintenance_Status'].mode()[0],
inplace=True)
df['Weather_Condition'].fillna(df['Ship_Type'].mode()[0], inplace=True)
```

После того, как заполнили пропуски, переходим к обработке выбросов. Выполним обработку уже известным методом IQR (Листинг 3).

Листинг 3 — Обработка выбросов

```
def remove_outliers_iqr(df, columns):
    df_clean = df.copy()
    for col in columns:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df_clean = df_clean[(df_clean[col] >= lower_bound) & (df_clean[col] <=
upper_bound)]
    return df_clean

numerical_cols = [
    'Speed_Over_Ground_knots', 'Engine_Power_kW', 'Distance_Traveled_nm',
    'Draft_meters', 'Cargo_Weight_tons', 'Operational_Cost_USD',
    'Revenue_per_Voyage_USD', 'Turnaround_Time_hours', 'Efficiency_nm_per_kWh',
    'Seasonal_Impact_Score', 'Weekly_Voyage_Count', 'Average_Load_Percentage'
]
df_no_outliers = remove_outliers_iqr(df, numerical_cols)
print(f"Было: {len(df)}, стало: {len(df_no_outliers)}")
df = df_no_outliers.reset_index(drop=True)
```

Выбросы успешно обработаны, переходим к кодировке категориальных признаков (Листинг 4).

Листинг 4 — Кодировка категориальных признаков

```
categorical_cols = ['Ship_Type', 'Route_Type', 'Engine_Type',
'Maintenance_Status', 'Weather_Condition']
le_dict = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    le_dict[col] = le
```

И выполним масштабирование числовых признаков. Код программы представлен в Листинге 5.

Листинг 5 — Масштабирование числовых признаков

```
df_cluster = df.drop(columns=['Date'])
X = df_cluster.copy()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
```

После того, как была проведена первичная предобработка данных, переходим к отбору признаков.

1.1 Задание 1. Отбор признаков по корреляции и простым критериям

Проведем отбор признаков, используя корреляционную матрицу, с целью обнаружения высокоррелированных признаков (Листинг 6).

Листинг 6 — Отбор признаков по корреляции

```
corr_matrix = X[numeric_features].corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape),
k=1).astype(bool))
high_corr = [col for col in upper.columns if any(upper[col] > 0.85)]
print("Высокоррелированные признаки (>0.85):", high_corr)
to_drop_corr = []
for col in high_corr:
    if col not in to_drop_corr:
        correlated_cols = upper.index[upper[col] > 0.85].tolist()
        if correlated_cols:
            to_drop_corr.append(correlated_cols[0])
to_drop_corr = list(set(to_drop_corr))
print("Будем удалять:", to_drop_corr)
X_no_corr = X.drop(columns=to_drop_corr)
```

Результат показал, что таких признаков не было обнаружено, это говорит о чистоте данных и минимальности мультиколлинеарности.

Теперь попробуем отобрать признаки методами SelectKBest (Листинг 7).

Листинг 7 — Отбор признаков по критериям

```
from sklearn.feature_selection import SelectKBest, f_classif,
mutual_info_classif
selector_f = SelectKBest(score_func=f_classif, k=10)
X_f = selector_f.fit_transform(X_no_corr, y)
selected_f = X_no_corr.columns[selector_f.get_support()].tolist()
selector_mi = SelectKBest(score_func=mual_info_classif, k=10)
X_mi = selector_mi.fit_transform(X_no_corr, y)
selected_mi = X_no_corr.columns[selector_mi.get_support()].tolist()
```

Продолжение Листинга 7

```
print("\nТоп-10 по f_classif:")
print(selected_f)
print("\nТоп-10 по mutual_info:")
print(selected_mi)
```

Получаем следующие результаты:

- Топ-10 по f_classif: ['Route_Type', 'Engine_Type', 'Distance_Traveled_nm', 'Draft_meters', 'Weather_Condition', 'Cargo_Weight_tons', 'Revenue_per_Voyage_USD', 'Seasonal_Impact_Score', 'Weekly_Voyage_Count', 'Average_Load_Percentage'];
- Топ-10 по mutual_info: ['Route_Type', 'Engine_Type', 'Weather_Condition', 'Cargo_Weight_tons', 'Operational_Cost_USD', 'Revenue_per_Voyage_USD', 'Turnaround_Time_hours', 'Seasonal_Impact_Score', 'Weekly_Voyage_Count', 'Average_Load_Percentage'].

Метод выделил смесь категориальных и числовых признаков, ориентируясь на линейные различия средних между классами эффективности.

Проверим устойчивость признаков (Листинг 8).

Листинг 8 — Устойчивость признаков

```
seeds = [0, 42, 123, 777]
sets_mi = []
for seed in seeds:
    sel = SelectKBest(score_func=lambda X, y: mutual_info_classif(X, y,
random_state=seed), k=10)
    sel.fit(X_no_corr, y)
    sets_mi.append(set(X_no_corr.columns[sel.get_support()])))
common = sets_mi[0]
for s in sets_mi[1:]:
    common = common & s
print("Устойчивые признаки (встречаются при всех random_state):")
print(list(common))
print("Количество:", len(common))
```

В результате выполнения блока кода было выявлено 8 устойчивых признаков. Это означает, что эти признаки надёжно связаны с целевой переменной и не являются случайными "шумовыми" отборами.

Сравним качество полученных моделей (Листинг 9).

Листинг 9 — Сравнение качества

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

model = RandomForestClassifier(n_estimators=100, random_state=42)
scores_full = cross_val_score(model, X_no_corr, y, cv=5, scoring='f1_weighted')
X_mi_df = X_no_corr[selected_mi]
scores_mi = cross_val_score(model, X_mi_df, y, cv=5, scoring='f1_weighted')
print("Сравнение качества (F1-weighted, 5-fold CV):")
print(f"Полные данные (без корреляций): {scores_full.mean():.4f} ± {scores_full.std():.4f}")
print(f"Отобранные (mutual_info): {scores_mi.mean():.4f} ± {scores_mi.std():.4f}")
```

Сравнение качества (F1-weighted, 5-fold CV):

- полные данные (без корреляций): 0.3302 ± 0.0082 ;
- отобранные (mutual_info): 0.3185 ± 0.0134 .

Качество модели снизилось незначительно, но в пределах стандартного отклонения. При этом размерность изменилась с 16 признаков до 10.

1.2 Задание 2. Сравнение методов отбора на ансамблевых моделях

Узнаем f1-weighted модели (Листинг 10).

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
rf_base = RandomForestClassifier(n_estimators=100, random_state=42)
base_score = cross_val_score(rf_base, X, y, cv=5, scoring='f1_weighted').mean()
print(f"Базовый F1 (все признаки): {base_score:.4f}")
```

Получаем значение, равное 0.3302. Это качество модели на полном наборе признаков.

Выделим 5 признаков по трем методам: Feature Importances, RFE и L1 (Листинг 11).

Листинг 11 — Выделение пяти признаков (FI)

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)
importances = rf.feature_importances_
feature_names = X.columns
top5_fi = [feature_names[i] for i in np.argsort(importances)[-5:][::-1]]
print("Топ-5 (Feature Importances):", top5_fi)
```


Получаем следующий список:

1. 'Revenue_per_Voyage_USD'.
2. 'Distance_Traveled_nm'.
3. 'Draft_meters'.
4. 'Turnaround_Time_hours'.
5. 'Cargo_Weight_tons'.

Теперь RFE (Листинг 12).

Листинг 12 — RFE

```
from sklearn.feature_selection import RFE
rfe = RFE(estimator=RandomForestClassifier(n_estimators=100, random_state=42),
n_features_to_select=1, step=1)
rfe.fit(X, y)
ranking = rfe.ranking_
top5_rfe = [feature_names[i] for i in np.argsort(ranking)[:5]]
print("Топ-5 (RFE):", top5_rfe)
```

Получаем следующий список:

1. 'Operational_Cost_USD'.
2. 'Turnaround_Time_hours'.
3. 'Revenue_per_Voyage_USD'.
4. 'Draft_meters'.
5. 'Average_Load_Percentage'.

Теперь L1 (Листинг 13).

Листинг 13 — L1

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(penalty='l1', solver='liblinear', C=0.1,
random_state=42)
lr.fit(X, y_encoded)
coef_abs = np.max(np.abs(lr.coef_), axis=0)
top5_l1 = [feature_names[i] for i in np.argsort(coef_abs)[-5:][::-1]]
print("Топ-5 (L1):", top5_l1)
```

Получаем список:

1. 'Engine_Type'.
2. 'Weather_Condition'.
3. 'Revenue_per_Voyage_USD'.
4. 'Distance_Traveled_nm'.

5. 'Weekly_Voyage_Count'.

Первый метод выделил основные операционные и экономические показатели: доход, расстояние, осадка, время и вес груза. Все они логично связаны с эффективностью рейса. Во втором добавился операционный расход и средняя загрузка, что говорит о том, что RFE более чувствителен к "затратной" составляющей эффективности. Третий метод выделил категориальные признаки (Engine_Type, Weather_Condition) и частоту рейсов (Weekly_Voyage_Count). Это показывает, что L1-регуляризация способна работать с "нелинейными" зависимостями через закодированные категории.

Построим график зависимости качества модели от числа признаков (Листинг 14).

Листинг 14 — График зависимости качества от числа признаков

```
def plot_feature_selection_curve(selector, X, y, method_name,
    scoring='f1_weighted'):
    scores = []
    n_features = []
    for n in range(1, X.shape[1] + 1):
        sel = selector(n_features_to_select=n)
        X_sel = sel.fit_transform(X, y)
        score = cross_val_score(rf_base, X_sel, y, cv=3, scoring=scoring).mean()
        scores.append(score)
        n_features.append(n)
    plt.plot(n_features, scores, label=method_name)
    return n_features, scores
plt.figure(figsize=(10, 6))
rfe_scores = []
for n in range(1, X.shape[1] + 1):
    rfe_temp = RFE(estimator=RandomForestClassifier(n_estimators=50,
        random_state=42), n_features_to_select=n, step=1)
    X_rfe = rfe_temp.fit_transform(X, y)
    score = cross_val_score(RandomForestClassifier(n_estimators=50,
        random_state=42), X_rfe, y, cv=3, scoring='f1_weighted').mean()
    rfe_scores.append(score)
plt.plot(range(1, X.shape[1]+1), rfe_scores, label='RFE')
importances = rf.feature_importances_
fi_scores = []
for n in range(1, X.shape[1] + 1):
    top_idx = np.argsort(importances)[-n:]
    X_fi = X.iloc[:, top_idx]
    score = cross_val_score(rf_base, X_fi, y, cv=3,
        scoring='f1_weighted').mean()
    fi_scores.append(score)
plt.plot(range(1, X.shape[1]+1), fi_scores, label='Feature Importances')
plt.xlabel('Количество признаков')
plt.ylabel('F1-weighted (3-fold CV)')
plt.title('Зависимость качества от числа признаков')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

График изображен на Рисунке 3.

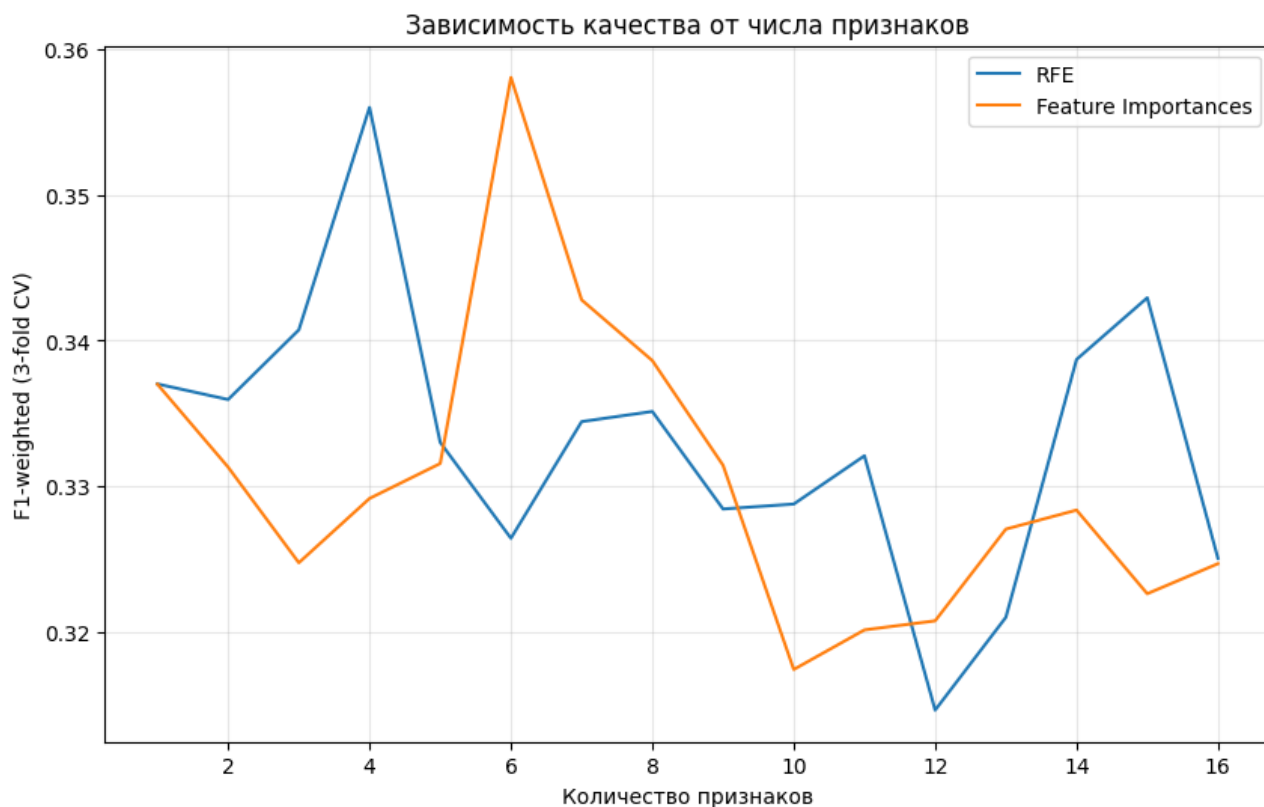


Рисунок 3 — Зависимость качества от числа признаков

На графике видим, что качество растет не монотонно. При 16 признаках качество модели падает, что указывает на переобучение или шум в полном наборе. Пиковое значение числа признаков равно 6.

Диаграмма Венна изображена на Рисунке 4.



Рисунок 4 — Диаграмма Венна

Всего был найден 1 общий признак между тремя методами.

1.3 Интерпретируемый отбор признаков

Наполним данные шумом (Листинг 15).

Листинг 15 — Шум

```
np.random.seed(42)
for i in range(1, 6):
    X[f'random_feature_{i}'] = np.random.randn(len(X))
print("Форма X после добавления шума:", X.shape)
```

Теперь в выборке 21 признак.

Выполним обучение catboost (Листинг 16).

Листинг 16 — CatBoost

```
from catboost import CatBoostClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model_cb = CatBoostClassifier(
    iterations=300,
    learning_rate=0.1,
    depth=6,
    random_seed=42,
    verbose=False
)
model_cb.fit(X_train, y_train)
y_pred = model_cb.predict(X_test)
f1_full = f1_score(y_test, y_pred, average='weighted')
print(f"F1-weighted (полная модель): {f1_full:.4f}")
```

В результате выполнения блока кода было получено значение 0.2952.

Оценим теперь значимость параметров с помощью permutation importance (Листинг 17).

Листинг 17 — Permutation Importance

```
from sklearn.inspection import permutation_importance
perm_imp = permutation_importance(
    model_cb, X_test, y_test,
    scoring='f1_weighted',
    n_repeats=5,
    random_state=42,
    n_jobs=-1
)
```

Продолжение Листинга 17

```
)
perm_indices = np.argsort(perm_imp.importances_mean)[:, :-1]
perm_features = X.columns[perm_indices]
perm_scores = perm_imp.importances_mean[perm_indices]
print("Permutation Importance (top-10):")
for i in range(10):
    print(f"{i+1}. {perm_features[i]}: {perm_scores[i]:.4f}")
```

Вот результат:

1. Weather_Condition: 0.0174.
2. Route_Type: 0.0129.
3. Revenue_per_Voyage_USD: 0.0103.
4. random_feature_2: 0.0027.
5. random_feature_4: -0.000.
6. Average_Load_Percentage: -0.000.
7. Engine_Type: -0.0018.
8. Engine_Power_kW: -0.0059.
9. Maintenance_Status: -0.0063.
10. Operational_Cost_USD: -0.0076.

Видим, что первые три признака наиболее важны, поскольку дальше идут шумовые. Permutation Importance корректно идентифицировал шумовые признаки — они имеют минимальный вклад.

Теперь попробуем shap-анализ (Листинг 18).

Листинг 18 — Shap-анализ

```
import shap
explainer = shap.TreeExplainer(model_cb)
shap_values = explainer.shap_values(X_test)
mean_abs_shap = np.mean(np.abs(shap_values), axis=(0, 2))
shap_indices = np.argsort(mean_abs_shap)[:, :-1]
shap_features = X.columns[shap_indices]
print("SHAP (top-10):")
for i in range(10):
    print(f"{i+1}. {shap_features[i]}: {mean_abs_shap[shap_indices[i]]:.4f}")
```

Вот результат:

1. Revenue_per_Voyage_USD: 0.0679.
2. Draft_meters: 0.0619.
3. random_feature_1: 0.0619.

4. random_feature_5: 0.0598.
5. Average_Load_Percentage: 0.0549.
6. Operational_Cost_USD: 0.0530.
7. random_feature_4: 0.0509.
8. random_feature_3: 0.0509.
9. Seasonal_Impact_Score: 0.0502.
10. Engine_Type: 0.0495.

Shap высоко оценил шумовые признаки, все дело в его особенности: он считает вклад каждого признака в каждое предсказание. Если шумовой признак случайно коррелирует с целевой переменной в обучающей выборке, SHAP может присвоить ему высокую важность. Это называется "overfitting to noise" — модель "запомнила" шум как сигнал.

Визуализируем результаты shap (Листинг 19).

Листинг 19 — Визуализация shap

```
shap.summary_plot(shap_values[:, :, 1], X_test, max_display=15)
```

График изображен на Рисунке 5.

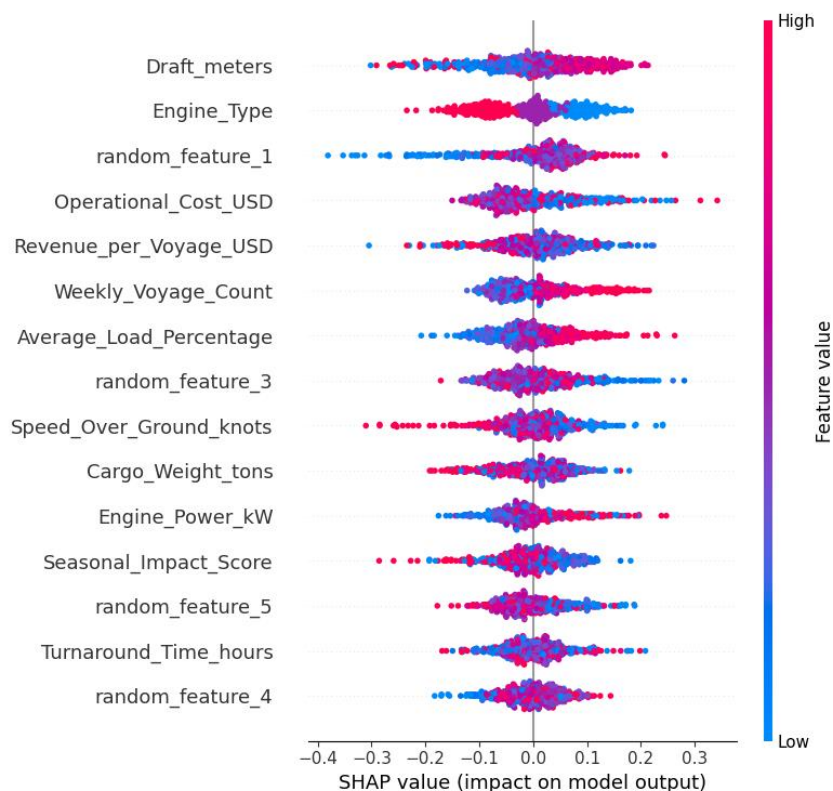


Рисунок 5 — Summary plot

На графике видно, что признаки отсортированы по важности. Шумовые признаки показывают случайное распределение значений, в то время как важные признаки демонстрируют нелинейную зависимость, например, высокие значения Revenue сдвигают прогноз в сторону класса High. Это подтверждает, что SHAP позволяет не только оценить важность, но и понять физический смысл влияния признаков.

ВЫВОД

В ходе выполнения практической работы №7 были успешно изучены методы отбора признаков в задачах машинного обучения, сравнение их эффективности и интерпретируемости, а также анализ их влияния на качество моделей.